

思维导图软件设计与实现

摘要：在当今数字化信息管理领域，思维导图作为一种发散性思维的可视化工具，对于知识整合、创意激发及项目管理具有不可替代的作用。本项目旨在设计并实现一款基于 Python 语言与 PyQt5 图形界面框架的轻量级桌面端思维导图编辑系统。报告详细阐述了从需求分析、系统架构设计、核心算法实现到最终测试验证的全过程。系统采用经典的 MVC（模型-视图-控制器）架构模式，实现了节点的多级拓扑管理、智能连线布局、富文本编辑及手写批注等核心功能。特别是在数据结构设计上，采用了多叉树与双向链表结合的模式，有效支撑了复杂的编辑操作与撤销重做机制。技术报告部分深入剖析了基于 QGraphicsScene 的图元渲染管线与坐标映射逻辑，并针对递归布局算法与内存管理进行了性能优化分析。测试结果表明，该系统具备良好的稳定性与交互体验，能够满足教育与办公场景下的基础需求。

关键词： 思维导图；Python；PyQt5；MVC 架构；递归布局；命令模式

小组分工情况

成员	分工
谢秋实 (队长)	1. 确定整体架构，选择库，推进软件开发过程，确定改进方向，对软件进行全功能测试 2. 软件中场景交互事件、文件管理、节点附加信息 UI 组件、语言切换与主题切换、部分命令、部分工具 3. 小组中期报告、小组软件设计报告的书写
朱拓源	1. 软件中节点属性及功能、手写绘图、连接线创建、部分组件、部分命令、部分工具 2. 完善代码底层的信号/槽通信
熊锦宸	1. 节点层级定义、按钮交互、预设模板、部分命令

目录

思维导图软件设计与实现	1
小组分工情况	2
一、 项目描述	5
1.1 项目背景与开发意义	5
1.2 需求分析	5
1.2.1 功能需求	5
1.2.2 非功能性需求	6
1.3 软硬件运行环境	6
1.4 开发工具与技术选型	7
二、 系统描述	7
2.1 系统总体架构	7
2.1.1 模型层 Model	7
2.1.2 视图层 View	8
2.1.3 控制层 Controller	8
2.2 核心交互机制设计	8
2.2.1 信号/槽机制	8
2.2.2 坐标系统	9
三、 数据结构描述	9
3.1 节点数据结构	9
3.2 连线数据结构	10
3.3 数据持久化模式	10
3.4 命令模式数据结构	11
四、 软件设计	11
4.1 核心类详细设计	11
4.1.1 Graph 类：场景管理	11
4.1.2 Node 类：节点图元	12
4.1.3 AnnotationItem 类：手写批注	12
4.2 辅助模块设计	13
4.2.1 Config 模块	13
4.2.2 Component 模块	13
4.3 关键算法逻辑	14

4.3.1 递归布局算法	14
4.3.2 深度优先搜索 DFS 与广度优先搜索 BFS	15
五、模块层次图	15
5.1 核心分层解析	15
5.1.1 UI 层：界面交互	15
5.1.2 命令层：逻辑控制	16
5.1.3 图元层：实体数据	16
5.2 辅助模块与入口	16
5.3 模块依赖总结	16
六、技术报告	17
6.1 核心功能	17
6.1.1 节点管理	17
6.1.2 连接线管理	17
6.1.3 文件操作	18
6.1.4 编辑功能	19
6.2 扩展功能实现	19
6.2.1 节点附加信息	19
6.2.2 手写批注	20
6.2.3 主题设置和语言切换	21
6.2.4 缩放和状态栏	21
6.2.5 预设主题	21
6.2.6 用户体验	22
6.3 系统测试与验证	22
七、总结与建议	23
7.1 工作总结与团队贡献	23
7.2 项目局限性分析	23
7.3 未来改进方向	23
八、参考文献	23

一、项目描述

1.1 项目背景与开发意义

随着知识经济的到来，信息呈现出碎片化与指数级增长的特征，如何高效地组织、管理与可视化复杂信息成为个人与企业面临的重要挑战。思维导图通过模拟人类大脑的放射性思维模式，将线性的文本信息转化为层级分明的图形网络，极大地提升了记忆效率与逻辑梳理能力。

本项目基于“软件工程课程设计”的要求，利用 Python 强大的生态系统与 PyQt5 高效的图形渲染能力，构建一个可扩展的思维导图编辑平台。这不仅是对图形用户界面 GUI 开发技术的实践，更是对数据结构、算法设计及设计模式的综合应用。

1.2 需求分析

1.2.1 功能需求

依照《软件课程设计》题目中的软件基本要求与可选要求，本系统确立了以下核心功能需求：

基本功能要求：

（1）节点管理：支持“中心主题-分支主题-子主题”的多级树状结构创建；支持删除指定节点及其所有后代节点；支持节点的自由拖拽移动，移动父节点时子树应跟随移动，且节点间的连接线需自动重新计算并绘制；提供基本的拷贝、剪切和粘贴功能。

（2）基础文本编辑：支持双击节点修改文本内容，确保文字能够正确显示在节点图元内部。提供给主题设置图标、链接、备注等功能。

（3）文件存取：建立标准的文件管理系统，支持新建空白导图，支持将当前编辑内容保存为 XML 格式的 .mm 文件，并能准确读取 XML 文件还原思维导图结构。

（4）界面交互：提供可视化的图形用户界面 GUI，包含主菜单、工具栏及绘图区域。

可选功能要求：

（1）预设模板库：支持多种图形结构的模板预设，包括鱼骨图、甘特图、流程图等，满足不同场景下的可视化需求。

（2）样式与富文本：支持节点视觉样式的自定义，包括修改背景填充颜色、文本颜色；支持字体加粗、斜体等富文本编辑。

（3）历史记录控制：实现基于命令模式的撤销与重做机制，支持对移动、编辑、删除等操作的回滚。

（4）子树剪贴板：支持以“子树”为单位进行复制、剪切和粘贴操作，便于内容的复用与重组。

（5）视图操作与可靠性：支持画布的 1%-200%无级缩放与平移导航；具备定时自动保存功能（默认 10 秒），防止意外退出导致数据丢失。

其他扩展功能：

（1）个性化主题与国际化：提供浅色、深色、黑色三种界面主题的实时切换；支持多语言切换，适应不同用户的使用习惯。

（2）丰富的辅助信息挂载：支持为节点挂载丰富的扩展信息，包括待办事项、详细备注、超链接以及图标。

（3）手写批注系统：提供手写涂鸦模式，允许用户使用画笔在画布上自由绘制标注，并提供橡皮擦功能擦除笔迹。

（4）多格式交互：支持将思维导图导出为 PNG 图片或 PDF 文档，支持调用系统打印机直接打印；支持解析 Markdown 文本并自动生成对应的思维导图结构。

1.2.2 非功能性需求

响应性：在节点数量达到 500 个时，界面缩放与拖拽的帧率应保持在 30 以上。

可靠性：具备自动保存机制，防止意外崩溃导致的数据丢失。

易用性：提供撤销与重做功能，快捷键需符合主流软件习惯。

1.3 软硬件运行环境

系统的开发与运行环境配置如下表 1-1 所示：

表 1-1 开发与运行环境配置表

类别	项目	详细描述/版本
硬件环境	处理器	Intel Core i7 和 Intel Core Ultra 9
	内存	8GB RAM 及以上
软件环境	操作系统	Windows 11 和 macOS

类别	项目	详细描述/版本
	开发语言	Python 3.7
	GUI 框架	PyQt5(5.15)
	IDE	PyCharm 和 VS Code
	版本控制	Git

1.4 开发工具与技术选型

本项目主要采用 Python 语言，核心依赖库及其作用如下：

PyQt5:作为核心 GUI 框架，利用其 Graphics View Framework 处理大规模图元的渲染、碰撞检测与事件分发。PyQt5 的信号与槽机制是实现模块解耦的关键。

xml.etree.ElementTree:用于处理 XML 格式的数据持久化，因其轻量级且内置于 Python 标准库，适合处理树状结构的序列化与反序列化。

re(Regular Expression):用于解析 Markdown 文本及富文本 HTML 标签的清洗与转换。

PyQt5.QtPrintSupport:提供跨平台的 PDF 生成与打印机接口支持。

二、系统描述

2.1 系统总体架构

本系统采用了经典的 MVC(Model-View-Controller)架构模式，结合 Qt 特有的场景-视图 Scene-View 机制，将数据逻辑、界面渲染与用户交互严格分离。各层级具体功能设计如下：

2.1.1 模型层 Model

Model 层负责维护思维导图的完整数据状态，且不直接依赖于任何界面组件。核心类设计如下：

Graph 类：作为核心数据容器，负责管理所有的节点（Node）与连线（Branch）。该类不仅存储基础数据，还负责场景内的空间索引管理，以优化检索效率。

Node 类：节点实体类。封装了节点的文本内容、样式属性、层级信息，以及指向父节点和子节点的引用关系。

Branch 类：连线实体类。定义节点间的连接关系，能够根据源节点和目标节点的坐标动态计算路径。

Annotation 类：负责管理手写批注的笔迹数据。

2.1.2 视图层 View

View 层负责将模型数据可视化呈现给用户，主要包含以下组件：

MainWindow 类：集成了菜单栏、工具栏和状态栏，主要负责处理文件级操作（如新建、保存）和全局设置。

CustomGraphicsView 类：作为观察 Graph 的视口，实现了滚轮缩放、右键画布拖拽以及橡皮擦模式下的光标变换等复杂交互，并通过坐标变换矩阵，负责将逻辑坐标精确映射为屏幕像素。

2.1.3 控制层 Controller

虽然 PyQt5 框架中没有显式的 Controller 类，但本系统通过信号与槽机制及命令模式实现了控制逻辑：

Command 模块：封装了 InsertNodeCommand、MoveCommand、EditCommand 等原子操作类，实现了操作的解耦。

QUndoStack：作为历史记录控制器，管理上述命令对象的入栈与出栈，从而实现标准的撤销（Undo）与重做（Redo）功能。

2.2 核心交互机制设计

本系统的交互流畅性主要依赖于 Qt 的事件处理系统与自定义的信号通信机制。

2.2.1 信号/槽机制

为了降低模块间的耦合度，系统广泛使用了信号槽机制进行通信。

当 Node 的位置发生变化时，会发射自定义信号 `nodeMoved`。连接该节点的 Branch 对象槽函数响应该信号，重新获取端点坐标并调用 `update()` 重绘路径。这种观察者模式确保了连线始终紧随节点，无需全局刷新。

MainWindow 发射 `themeChanged` 信号时，Graph 接收信号并遍历所有图元，调用其 `updateStyle()` 方法，实现界面的即时换肤。

2.2.2 坐标系统

系统涉及三个关键坐标系：

- 1. 图元坐标 Item Coordinates：以每个节点中心为原点(0,0)，用于处理节点内部的点击测试和局部绘制。
- 2. 场景坐标 Scene Coordinates：逻辑上的无限大画布，所有节点在场景中都有唯一的绝对坐标(x,y)。序列化保存时存储此坐标。
- 3. 视图坐标 View Coordinates：用户屏幕上的像素坐标。QGraphicsView 负责处理缩放和平移变换，将场景坐标映射到视图坐标。例如，在手写批注时，必须将鼠标点击的视图坐标转换为场景坐标，以确保笔迹位置在缩放后依然准确。

三、数据结构描述

思维导图本质上是一种树状数据结构，但在图形编辑场景下，需要额外维护空间属性与状态信息。

3.1 节点数据结构

节点是系统的核心实体。为了支持高效的增删改查与渲染，Node 类设计了包含拓扑关系、视觉属性与内容数据的复合结构。其核心属性定义如下表 3-1 所示：

表 3-1 节点核心属性定义表

属性名称	类型	描述与用途
parentNode	Node	指向父节点的引用，根节点为 None。用于向上回溯路径。
sonNode	List[Node]	存储所有子节点的列表。用于广度优先搜索及递归布局。
m_level	int	节点层级：1-中心主题，2-分支主题，3-子主题，4-自由主题。决定默认样式。
x, y	float	节点在场景中的逻辑坐标。
m_defaultText	str	纯文本内容，用于搜索和简略显示。
htmlContent	str	HTML 格式的富文本内容，支持字体样式持久化。
m_note	str	节点的隐藏备注信息。
m_link	str	超链接地址。

属性名称	类型	描述与用途
m_color	QColor	背景填充颜色。
m_textColor	QColor	文本颜色。
m_editable	bool	标记节点是否处于可编辑状态，防止误操作。
m_isExpanded	bool	标记子树是否展开。

此数据结构不仅维护了逻辑树，还通过 x,y 维护了空间树。这种双重结构使得系统既能快速进行逻辑遍历，又能高效进行空间查询。

3.2 连线数据结构

Branch 对象实质上是两个节点之间的关系投影。它不独立存储复杂的几何路径，而是动态依赖于两端节点。

源节点 srcNode: 连线的起始端，通常为父节点。

目标节点 dstNode: 连线的终止端，通常为子节点。

样式属性 width,color: 定义线条的视觉风格。

动态计算逻辑: Branch 覆写了 paint() 方法，每次重绘时实时读取 srcNode.pos() 和 dstNode.pos()，计算贝塞尔曲线控制点，从而绘制出平滑曲线。

3.3 数据持久化模式

为了保证数据的可移植性，系统采用 XML 格式存储。存储策略采用深度优先遍历，这保证了文件结构与思维导图的视觉层级一致。

XML 文件结构如下：

```
<data>
<node x="0"y="0"m_level="1"htmlContent="中心主题"m_note=""m_link="">
<node x="200"y="-50"m_level="2"htmlContent="分支主题 1">
<node x="400"y="-50"m_level="3"htmlContent="子主题 A"/>
</node>
<node x="200"y="50"m_level="2"htmlContent="分支主题 2"/>
</node>
<annotation type="path"points="100,100;105,102;..."color="#FF0000"width="2"/>
</data>
```

这种嵌套的 XML 结构直观地表达了包含关系，且利用 XML 的属性存储元数据，利

用子元素存储层级关系，解析时可直接递归重建对象树。

3.4 命令模式数据结构

为了支持撤销/重做，系统维护了一个操作栈。每个栈帧是一个 Context 对象，记录操作前后的状态差量 Delta。

快照策略：对于移动操作，仅记录移动前后的坐标向量\$(dx,dy)\$；对于属性修改，记录修改前后的值。

对象生命周期：在“删除节点”命令中，被删除的节点对象并不会立即从内存销毁，而是被 RemoveNodeCommand 持有引用。当执行“撤销删除”时，直接将该对象重新加入场景，保留了所有子节点及属性状态。

四、软件设计

4.1 核心类详细设计

本系统基于面向对象的设计原则，将思维导图的业务逻辑抽象为场景 Graph、图元 Node 与辅助装饰 AnnotationItem 三个核心实体类，它们共同构成了模型层的骨架。

4.1.1 Graph 类：场景管理

Graph 类是整个思维导图数据的容器与交互的中枢。它不仅负责维护图元的生命周期，还承担着节点间的信号路由与布局调度任务。在节点的创建过程中，Graph 采用工厂模式统一管理节点的实例化与信号连接，如下述代码所示：

```
def nodeFactory(self):
    node = Node()
    node.setTheme(self.m_theme)
    node.setScale(2.8)
    # 连接节点的各种信号到场景的槽函数，实现节点与场景的交互
    node.nodeChanged.connect(self.nodeChanged)
    node.nodeEdited.connect(self.nodeEdited)
    node.nodeSelected.connect(self.nodeSelected)
    node.nodeMoved.connect(self.nodeMoved)
    return node
```

通过 nodeFactory 方法，系统在创建节点的同时完成了 nodeChanged、nodeMoved 等关键信号的绑定。这种设计确保了当单个节点的状态发生变化时，Graph 能够立即捕获并做出响应，例如触发 adjustBranch() 重新计算连接线路径，或触发 contentChanged 信号通知主窗口更新保存状态，从而实现了图元与场景之间的高内聚、低耦合交互。

4.1.2 Node 类：节点图元

Node 类是用户直接交互的核心实体。它并没有简单地使用标准的矩形控件，而是实现高度自定义的视觉效果。为了适应不同的层级和主题风格，Node 类内部实现了复杂的渲染逻辑：

```
def paint(self, painter, option, w):
    rect = self.boundingRect()
    # 根据主题和节点层级设置不同的颜色渐变
    if self.m_colorCustomized:
        gradient = QLinearGradient(rect.topLeft(), rect.bottomRight())
        # ... (省略颜色计算逻辑)
        painter.setBrush(QBrush(gradient))
        painter.setPen(QPen(QBrush(border_color), border_width, Qt.SolidLine,
Qt.RoundCap, Qt.RoundJoin))
        painter.drawRoundedRect(rect, 12.0, 6.0) # 更大的圆角，更可爱

    # 绘制选中状态的阴影和缩放手柄
    if self.isSelected():
        # ... (绘制阴影逻辑)
        painter.drawEllipse(handle_rect) # 绘制右下角缩放手柄
```

这段代码展示了 Node 类的绘制管线。系统首先根据当前的主题配置和节点层级动态计算渐变色与边框样式，使用 `drawRoundedRect` 绘制出圆润的节点背景。特别值得注意的是，代码中还集成了状态检测逻辑，当 `isSelected()` 为真时，会额外绘制阴影效果及右下角的缩放手柄，将交互反馈直接融合在渲染循环中，既保证了视觉的一致性，又提升了操作的直观感。

4.1.3 AnnotationItem 类：手写批注

为了实现模拟纸笔的手写体验，AnnotationItem 类继承自 `QGraphicsPathItem`。该类的核心设计在于如何处理笔迹的层级关系以及路径的渲染。

```
class AnnotationItem(QGraphicsPathItem):
    def __init__(self, path=None, pen=None, *args, **kwargs):
        super(AnnotationItem, self).__init__(*args, **kwargs)
        if path: self.setPath(path)
        # ...
        # 批注项应该在节点之上，但低于选择框
        self.setZValue(100)
        self.setFlag(QGraphicsItem.ItemIsSelectable, True)
        self.setFlag(QGraphicsItem.ItemIsMovable, False)
```

代码中的 `self.setZValue(100)`是实现“涂鸦”效果的关键。在 Qt 的图形视图框架中，Z 值决定了图元的堆叠顺序。将批注的 Z 值设为 100 高于节点，确保了无论节点如何移动或重绘，手写笔迹始终悬浮于节点内容之上，真实还原了在纸张表面书写的物理层级。同时，禁用了 `ItemIsMovable` 属性，防止用户在浏览导图时意外拖动笔迹，保证了数据的稳定性。

4.2 辅助模块设计

系统的可维护性与扩展性很大程度上依赖于辅助模块的设计。`Config` 模块与 `Component` 模块分别承担了全局配置管理与 UI 组件封装的职责。

4.2.1 Config 模块

`Config` 模块作为一个静态配置中心，通过定义全局常量来规范系统行为。这种设计使得系统的层级定义与视觉规范统一管理。

```
# 节点层级定义
MainThemeLevel = 1      # 中心主题层级
SecondThemeLevel = 2    # 分支主题层级
ThirdThemeLevel = 3     # 子主题层级
# 窗口尺寸配置
NOTE_SIZE = (300, 250)  # 备注窗口的尺寸
AUTOSAVE_TIME = 10000   # 自动保存时间间隔
```

如代码所示，通过将 `MainThemeLevel`、`AUTOSAVE_TIME` 等关键参数提取为常量，开发人员可以在不修改业务逻辑代码的情况下，仅通过调整 `Config` 文件即可改变系统的层级逻辑或自动保存策略。这不仅提高了代码的可读性，也为后续的功能迭代提供了极大的便利。

4.2.2 Component 模块

`Component` 模块体现了界面逻辑与业务逻辑分离的思想。以待办事项列表为例，系统将其封装为一个独立的 `renderTodoListToPixmap` 功能，将复杂的列表控件转换为轻量级的图像资源。

```
def renderTodoListToPixmap(self):
    # 计算所需尺寸
    font_metrics = QFontMetrics(font)
    # ... (计算 max_width 和 total_height)

    # 创建图片并绘制
```

```
pixmap = QPixmap(max(max_width, 300), total_height)
painter = QPainter(pixmap)
# ... (绘制背景、标题、复选框状态)
return pixmap
```

如代码所示，通过将 `MainThemeLevel`、`AUTOSAVE_TIME` 等关键参数提取为常量，开发人员可以在不修改业务逻辑代码的情况下，仅通过调整 `Config` 文件即可改变系统的层级逻辑或自动保存策略。这不仅提高了代码的可读性，也为后续的功能迭代（如新增层级样式）提供了极大的便利。

4.3 关键算法逻辑

思维导图软件的核心难点在于如何自动化地管理节点布局以及处理复杂的数据结构。本系统重点实现了递归布局与基于搜索的子树管理算法。

4.3.1 递归布局算法

为了防止节点在展开或增加子节点时发生重叠，系统实现了一套自底向上的递归调整算法。当一个节点的位置或尺寸发生变化时，`adjustNode` 方法会被触发，以确保兄弟节点之间保持足够的间距。

```
def adjustNode(self, parent, son, reverse=False):
    # 遍历父节点的所有子节点
    for node in parent.sonNode:
        if node == son: continue
        # 如果其他子节点在新子节点的上方，向下移动
        if node.y < son.y:
            self.moveTree(node, sign * (node.m_margin + node.m_size[1]) / 2)
        # 如果其他子节点在新子节点的下方，向上移动
        elif node.y > son.y:
            self.moveTree(node, -1 * sign * (node.m_margin + node.m_size[1])
/ 2)

    # 如果父节点还有父节点，递归调整
    if parent.parentNode:
        self.adjustNode(parent.parentNode, parent, reverse)
```

这段代码清晰地展示了布局调整的传播机制。算法首先检查当前父节点下的所有子节点，如果基于 Y 坐标判断发现空间不足，则调用 `moveTree` 将相邻的兄弟节点及其整个子树进行垂直平移。关键在于最后的递归调用 `self.adjustNode(parent.parentNode, ...)`，它确保了这种调整不仅仅局限于当前层级，而是会像波纹一样向上传播至根节点，从而实现了智能自动布局。

4.3.2 深度优先搜索 DFS 与广度优先搜索 BFS

在处理“移动子树”、“删除节点”或“复制粘贴”等批量操作时，系统需要准确地获取某个节点下的所有后代节点。为此，系统在 `getSubTree` 方法中采用了基于队列的广度优先搜索 BFS 策略。

```
def getSubTree(self, node):
    subTree = []
    queue = []
    queue.insert(0, node) # 将起始节点加入队列

    while queue:
        v = queue.pop()    # 取出节点
        subTree.append(v)
        # 将该节点的所有子节点加入队列
        for sonNode in v.sonNode:
            queue.insert(0, sonNode)
    return subTree
```

虽然思维导图在视觉上是树状结构，但在内存管理中，仅仅依靠递归往往会导致栈溢出风险。这里引用的代码通过维护一个 `queue` 列表，迭代地将当前层级的所有子节点加入待处理队列，从而高效地遍历出完整的子树结构。这种非递归的 BFS 实现方式不仅稳健，而且返回的线性列表 `subTree` 非常适合后续的批量坐标变换（如 `moveTree` 方法中的遍历）或 XML 序列化操作。

五、模块层次图

本系统的模块设计严格遵循“高内聚、低耦合”的软件工程原则，采用分层架构将系统划分为界面展示、逻辑控制与图元实体三个核心层次。各层级之间通过明确定义的接口与信号机制进行交互，确保了系统的可维护性与扩展性。图 5-1 展示了系统各模块文件之间的依赖与调用关系。

5.1 核心分层解析

根据功能职责的不同，系统主要划分为以下三个逻辑层级：

5.1.1 UI 层：界面交互

UI 层主要由 `mainwindow.py` 和 `component.py` 组成，作为用户直接感知的交互界面，承担着展示与基础交互的职责。其中 `mainwindow.py` 作为主窗口容器，集成了菜单、工具栏等全局控件，负责处理文件级操作并将具体业务逻辑转发至下层；而 `component.py` 则封装了各类独立的辅助 UI 组件，通过与主窗口的双向交互，实现属性设置与参数反馈的

闭环。

5.1.2 命令层：逻辑控制

命令层包含 `graph.py` 与 `command.py`，它们构成了连接 UI 层与底层图元的逻辑枢纽，起到了类似 MVC 架构中控制器的作用。`graph.py` 作为核心模型管理器维护所有图元的生命周期并分发用户事件；`command.py` 则基于命令模式将移动、删除、编辑等操作封装为独立对象，与 `Graph` 模块紧密配合，从而支撑起系统完善的撤销与重做机制。

5.1.3 图元层：实体数据

图元层是系统的最小执行单元，包含 `Node.py`、`Branch.py` 和 `Annotation.py` 三个模块，它们在 `graph.py` 的统一聚合管理下负责具体的渲染与数据存储。该层级明确了职责划分：`Node.py` 定义节点的文本内容与样式属性，`Branch.py` 负责动态计算节点间的连接路径，而 `Annotation.py` 则专门处理手写批注的笔迹渲染，共同构建了思维导图的可视化实体。

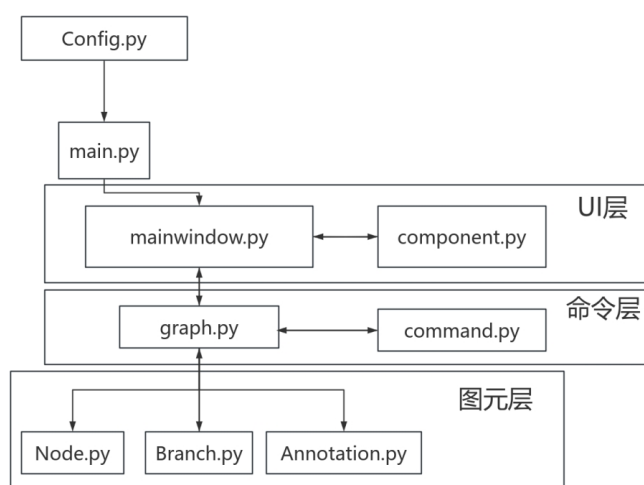


图 5-1 模块依赖关系层次图

5.2 辅助模块与入口

除了上述核心业务分层外，系统还包含两个基础支撑模块：`Config.py` 与 `main.py`。`Config.py` 处于依赖链的底层，作为一个独立的静态配置模块，它不依赖任何业务逻辑，仅提供全局通用的常量定义供各层调用以确保规范统一；`main.py` 则是整个系统的启动入口，负责初始化 `QApplication` 环境、加载配置信息并实例化主窗口，从而引导应用程序正式运行。

5.3 模块依赖总结

通过图 5-1 的模块依赖关系可以看出，系统的数据与控制流向呈现清晰的自上而下结

构：由 main.py 启动 UI 层，UI 层捕获操作后调用命令层进行逻辑处理，最终驱动图元层更新视图状态。这种设计不仅理清了调用顺序，更实现了同层级模块间的横向解耦，使得未来无论是替换 UI 主题还是优化底层渲染算法，都能将代码修改的影响范围控制在最小单元内。

六、技术报告

6.1 核心功能

6.1.1 节点管理

- (1) 通过按键可以创建、删除、修改各级主题节点；
- (2) 支持将节点修改为中心主题、分支主题、子主题和自由主题；
- (3) 双击节点文本可输入不等长文字信息；
- (4) 右键节点出现的编辑功能列表可修改节点颜色和文本颜色自定义；
- (5) 左键拖拽节点进行移动，移动子树模式下该节点将与其子树一起移动。

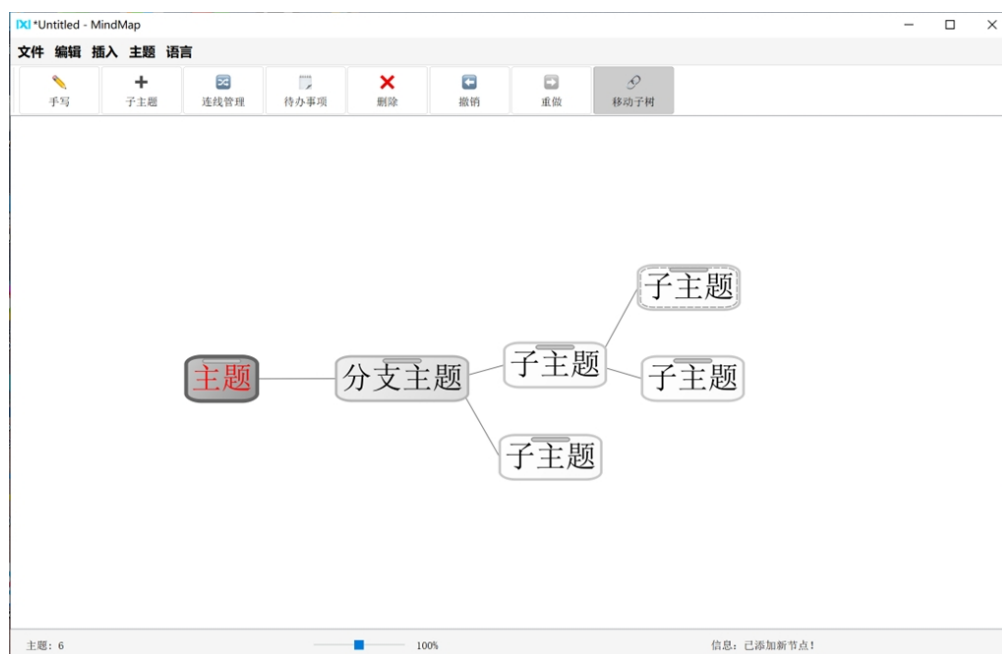


图 6-1 节点管理

6.1.2 连线管理

- (1) 添加新节点时自动生成父子节点之间的连接线；
- (2) 支持手动管理连接线，框选节点并点击连线管理，可创建或删除其间的连线；

(3) 移动节点时，连线自动跟随节点位置调整。

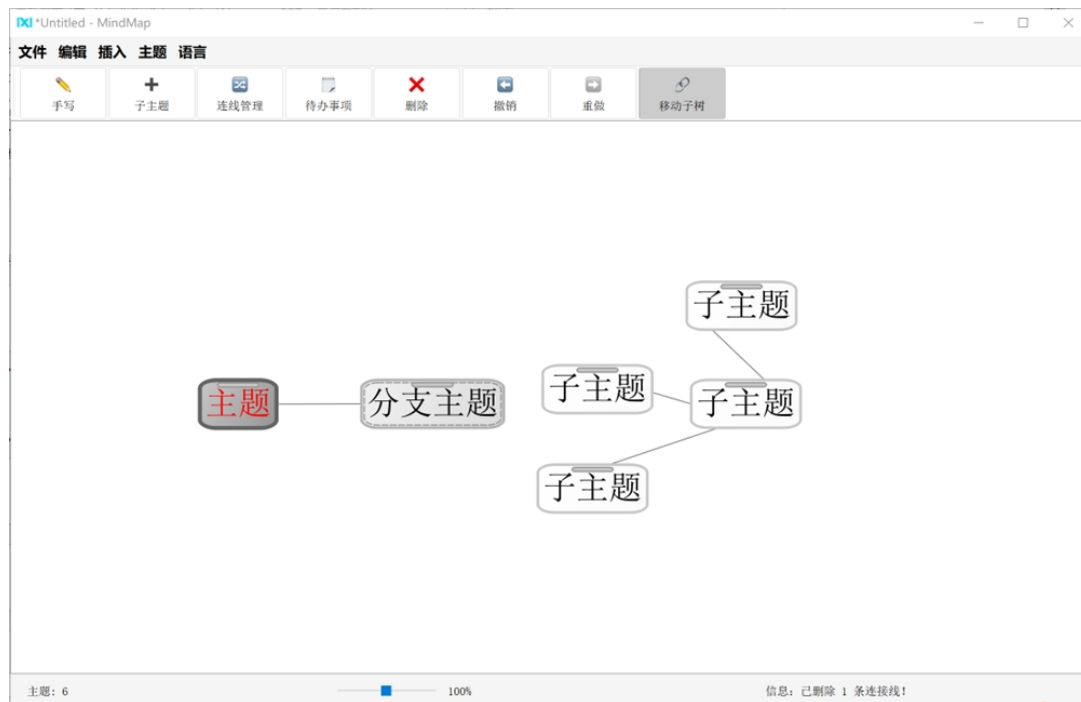


图 6-2 连接线管理

6.1.3 文件操作

- (1) 在文件栏中可新建、打开、保存思维导图文件为 XML 格式的 .mm 文件；
- (2) 每 10 秒间隔，文件将自动保存功能；
- (3) 支持导出为 PNG 图片格式或 PDF 文档格式；
- (4) 有打印功能；
- (5) 支持从 Markdown 文件导入为思维导图。

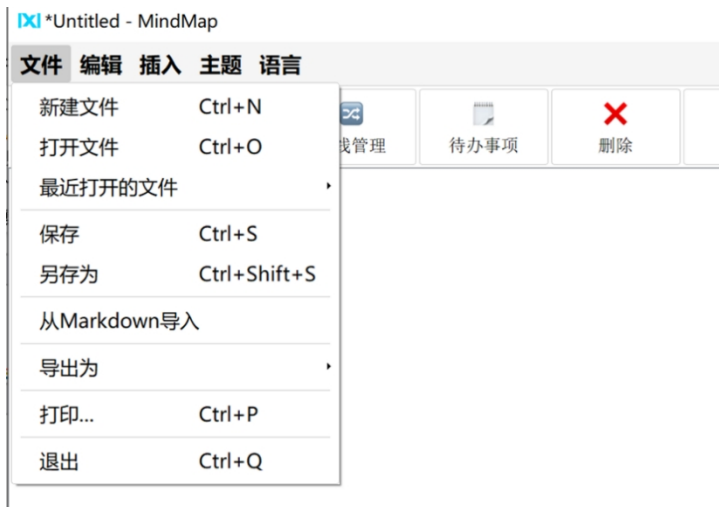


图 6-3 文件操作

6.1.4 编辑功能

- (1) 每一步操作都支持撤销和重做；
- (2) 可以复制、剪切、粘贴选定节点；
- (3) 可以删除节点且支持删除子树；
- (4) 所有操作都有快捷键，编辑栏标注出了各操作的快捷键。



图 6-4 编辑功能

6.2 扩展功能实现

6.2.1 节点附加信息

在插入中，可在节点中添加备注信息、可点击跳转的超链接、文本批注、图标、显示完成状态的待办事项表。



图 6-5 节点附加信息

6.2.2 手写批注

- (1) 支持手写绘制批注，可切换笔模式和橡皮擦模式；
- (2) 支持自定义画笔颜色和粗细；
- (3) 在橡皮擦模式下左键框选可删除批注。

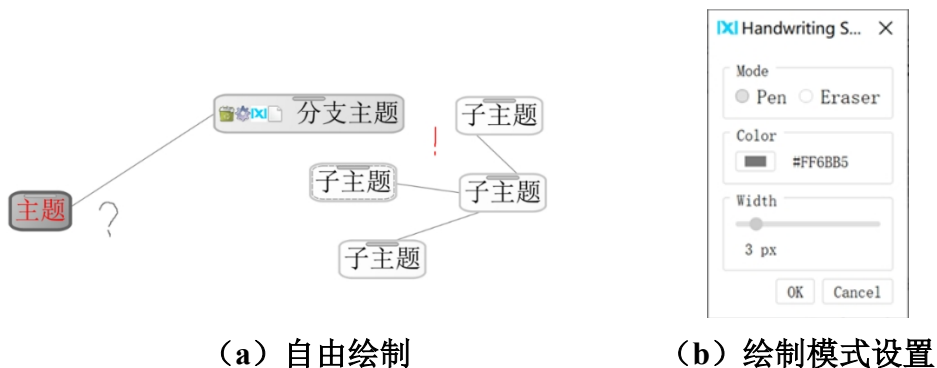
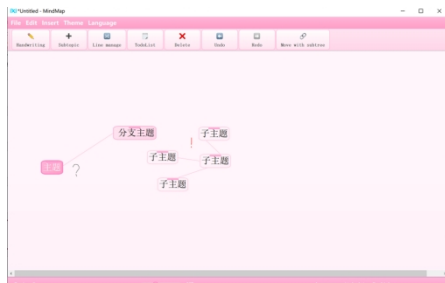


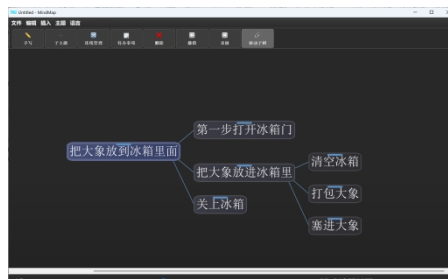
图 6-6 手写批注

6.2.3 主题设置和语言切换

- (1) 界面提供三种颜色主题可切换：浅色、深色、黑白；
- (2) 有双语支持可切换界面语言为中文或英文；



(a) 浅色+英文



(b) 深色+中文

图 6-7 主题设置和语言切换

6.2.4 缩放和状态栏

- (1) 支持通过拖动滑块进行界面的放大和缩小；
- (2) 下方状态栏显示当前节点数和操作提示。

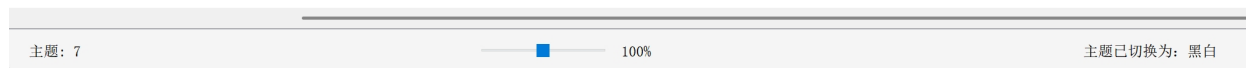


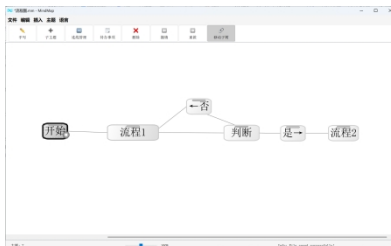
图 6-8 缩放和状态栏

6.2.5 预设主题

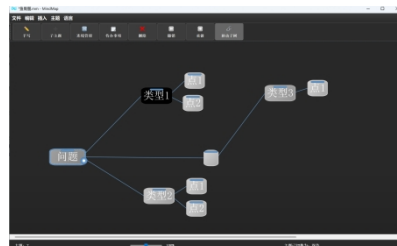
指出甘特图、流程图、鱼骨图三种预设主题：



(a) 甘特图



(b) 流程图



(c) 鱼骨图

图 6-9 预设主题

6.2.6 用户体验

该软件具有友好的图形操作界面；可左键拖转用虚线框选多节点进行操作；右键出现菜单进行快捷操作；上方显示工具栏快捷按钮。

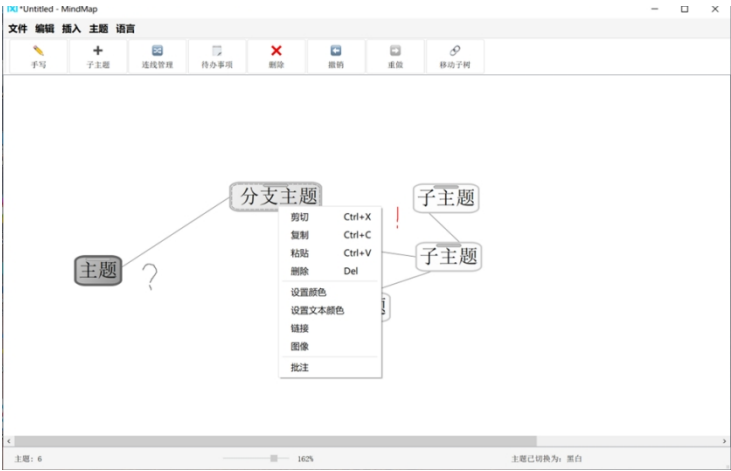


图 6-8 用户体验

6.3 系统测试与验证

系统经过了详细的功能测试，部分测试数据如下表 6-1 所示：

表 6-1 核心功能测试用例及结果

测试 ID	测试模块	测试内容	预期结果	实际结果	测试结论
T-001	节点管理	连续创建 5 层节点	节点自动排列，连线正确	节点排列整齐，无重叠	通过
T-002	撤销重做	移动节点后按 Ctrl+Z	节点瞬间回到原位	节点回到原位，连线跟随	通过
T-003	文件保存	保存含中文字符的导图	生成 UTF-8 编码 XML	XML 解析正常，无乱码	通过
T-004	手写批注	快速绘制复杂曲线	线条流畅，无明显延迟	线条平滑，Z 轴覆盖正常	通过
T-005	性能测试	加载 1000 个节点	打开时间<2s	打开耗时 1.8s，	基本通过

测试 ID	测试模块	测试内容	预期结果	实际结果	测试结论
		文件		缩放略卡顿	

七、总结与建议

7.1 工作总结与团队贡献

本项目历时数周，团队成员分工协作，圆满完成了课程设计任务。

组为组长，我见证了我们小组的三个人在这次软件课程设计中，一步一步从“代码初学者”变成了会用 Anaconda 管理项目、PyQt5 库创建图形界面、Git 进行版本控制的“代码生产者”。虽然思维导图是一个小任务，但是我们在解决一个个困难的过程中学到了很多。

从初期的查阅资料、中期第一次点亮 GUI 界面、中后期不断添加新功能、后期的大量测试与修复 bug，我们对 Python 语言和软件工程的设计思想有了越来越多的理解。

正是三个人的默契配合，才能在最后完成所有基础要求、所有可选要求、并做出了 Todolist、Markdown 导入、三种主题风格切换、手写批注等自由发挥的功能。

7.2 项目局限性分析

尽管系统功能已初具规模，但仍存在以下局限：

大文件性能瓶颈：当节点数超过 2000 时，XML 解析与全场景渲染会导致显著卡顿。

自由度限制：虽然支持连线管理，但连接线属于不具有父子关系的“非关联线”，该功能不符合思维导图的直觉，有待改进。

7.3 未来改进方向

针对上述问题，提出以下改进建议：

异步 I/O：引入多线程技术，将文件读写操作移至后台线程，配合进度条显示，确保主界面永不卡死。

协同编辑：探索引入 WebSocket 通信，将 XML 数据模型转化为 JSON 差异包，实现局域网内的多用户实时协同编辑功能。

八、参考文献

[1] 复旦大学图书馆. 思维导图工具 XMind 的使用

- <https://library.fudan.edu.cn/upload/article/files/59/8d/0a3a3ac84b15918abdb6721d8077/dff1fc26-0828-4148-9caf-bb063eba1953.pdf>.
- [2] Python GUIs. QGraphics vector graphics interfaces with Python and PyQt6
<https://www.pythonguis.com/tutorials/pyqt6-qgraphics-vector-graphics/>.
 - [3] Esveo. Undo, Redo, and the Command Pattern
<https://www.esveo.com/en/blog/undo-redo-and-the-command-pattern/>.
 - [4] The Qt Company. Model/View Programming
<https://doc.qt.io/qt-6/model-view-programming.html>.
 - [5] Real Python. Model-View-Controller (MVC) in Python Web Apps: Explained With Lego
<https://realpython.com/lego-model-view-controller-python/>.
 - [6] ResearchGate. EksPy: a new Python framework for developing graphical user interface based PyQt5
https://www.researchgate.net/publication/377878667_EksPy_a_new_Python_framework_for_developing_graphical_user_interface_based_PyQt5.
 - [7] The Qt Company. Graphics View Framework
<https://doc.qt.io/qt-6/graphicsview.html>.
 - [8] Lim R. Algorithm for Drawing Trees
<https://rachel53461.wordpress.com/2014/04/20/algorithm-for-drawing-trees/>.